

Serverless Function Cost and Performance Predictor

An Analysis of Azure Functions Workload Data for Cost Estimation Using Machine Learning

Prepared by

Muhammad Fiaz

Project Report

2026

Abstract

This report presents an end to end analysis of serverless computing workload data drawn from the Azure Functions 2019 public trace released by Microsoft. The study examines invocation patterns across more than six hundred thousand functions, engineers a cost estimation metric grounded in common serverless billing principles, and develops a machine learning model to predict function level cost from invocation behavior. The analysis places particular emphasis on methodological rigor, including the identification and removal of a data leakage issue that would otherwise have produced misleading results. The final model, trained exclusively on invocation frequency and trigger type, explains close to half of the variance in estimated cost, a result that is both realistic and practically meaningful. The project demonstrates the application of data analysis, feature engineering, and predictive modeling techniques to a problem central to cloud resource management and performance modeling.

1. Introduction

Serverless computing has become a widely adopted model for deploying cloud applications, allowing developers to run code without provisioning or managing servers directly. In this model, cloud providers charge based on how frequently a function is invoked, how long each invocation takes to execute, and how much memory is allocated to it. Understanding these cost drivers is important for developers and organizations seeking to manage cloud expenditure and plan deployments efficiently.

This project analyzes a publicly available dataset of real Azure Functions invocations to understand workload characteristics and to determine whether the cost associated with a function can be estimated from its invocation pattern alone, before detailed execution data is available. The analysis is structured around five objectives: loading and exploring the invocation data, analyzing invocation frequency patterns, engineering features related to cost and performance, building a predictive model, and summarizing the overall findings.

2. Dataset Description

The dataset used in this project is the Azure Functions Dataset 2019, released publicly by Microsoft for research and academic use as part of the Azure Public Dataset repository. It captures a subset of the workload running on the Azure Functions platform in July 2019 and consists of three related files.

File	Description	Original Size
invocations_per_function_md.feather	Minute level invocation counts for each function over a fourteen day period	651 MB
function_durations_percentiles.feather	Execution duration statistics per function, recorded daily	118 MB
app_memory_percentiles.feather	Memory allocation statistics per application, recorded daily	29 MB

The raw invocation file contains one thousand four hundred and forty minute level columns representing a full day of activity, which made direct loading in a standard pandas workflow impractical within the available memory. This was addressed using PyArrow for memory mapped file access and DuckDB for in-process aggregation, allowing the wide time series data to be summarized into function level statistics without exceeding the available system memory.

3. Methodology

3.1 Data Loading and Aggregation

The invocation dataset was aggregated at the function level to produce three summary metrics for each function: total invocations across the observed period, average invocations per minute, and peak invocations per minute. This transformation reduced the dataset from a wide, memory intensive format to a compact table of six hundred and eighteen thousand five hundred and forty five functions, each described by a small number of interpretable metrics.

3.2 Exploratory Analysis

Exploratory analysis examined the distribution of trigger types, the volume of invocations across functions, and the structure of applications in terms of function count. Timer and HTTP triggers were found to account for over seventy percent of all functions, while queue, orchestration, event, and storage triggers made up the remainder. Invocation volume was found to be heavily right skewed, with a median of forty four invocations per day and a maximum exceeding one hundred and eighty million, indicating that a small number of functions are responsible for the majority of overall workload.

A similar pattern was observed at the application level. The dataset contains twenty four thousand two hundred and seventy four applications, with a median of fourteen functions per application, though some applications contain several thousand functions. This concentration of activity in a small subset of applications and functions is consistent with patterns reported in prior characterizations of serverless workloads.

3.3 Feature Engineering

To connect invocation behavior with cost, the invocation summary was merged with the duration and memory datasets using function and application identifiers. Approximately two percent of records lacked corresponding memory data and were removed, resulting in a final merged dataset of five hundred and ninety nine thousand and eighty rows.

A cost proxy metric was then constructed by combining memory allocation, execution duration, and invocation frequency, following the general principle used by cloud providers to bill serverless usage. The metric was computed as the product of average memory in gigabytes, average duration in seconds, and total invocations, producing a measure referred to as estimated cost proxy. Additional performance

indicators were also derived, including duration variability and the ratio between the ninety ninth and fiftieth percentile of execution duration.

During validation, forty two records were found to contain negative duration values due to errors in the original trace data. These records, representing less than one hundredth of one percent of the dataset, were removed to preserve the mathematical validity of the engineered features.

3.4 Model Development

An initial Random Forest regression model was trained using invocation, duration, and memory features to predict the log transformed cost proxy. This model achieved a very high R-squared value of 0.9993, a result that on closer inspection indicated data leakage, since the target variable had been directly constructed from the duration and memory features included in the model. Because these features would not realistically be known in advance for a function under evaluation, the model was revised.

The feature set was rebuilt to include only information available from invocation behavior: total invocations, average invocations per minute, maximum invocations per minute, and trigger type, encoded using one hot representation. The target variable remained the log transformed cost proxy. A new Random Forest regressor was trained on this restricted feature set and evaluated on a held out test set comprising twenty percent of the data.

4. Results

The revised model achieved an R-squared of 0.4785, a mean absolute error of 1.3125, and a root mean squared error of 1.6981, all computed in log space. This indicates that invocation frequency and trigger type alone explain nearly half of the variance in estimated cost, a realistic outcome given that execution duration and memory allocation, both significant cost contributors, were deliberately excluded from the feature set.

Feature importance analysis showed that total invocations and average invocations per minute together accounted for approximately seventy nine percent of the model's predictive weight, confirming that invocation volume is the dominant available signal for cost estimation. Trigger type contributed modestly, with queue and timer based functions showing slightly greater influence than other trigger categories. Examination of predicted against actual values showed reasonable alignment at lower cost levels, with the model tending to underpredict the small number of extremely high cost functions, suggesting that factors beyond invocation frequency contribute meaningfully to cost at the upper end of the distribution.

5. Discussion

The results of this project illustrate two points relevant to cloud workload analysis. First, serverless workloads are highly uneven, with a small number of functions and applications responsible for a disproportionate share of total invocation volume and cost. This concentration has direct implications

for how infrastructure and monitoring resources should be prioritized. Second, invocation frequency alone provides a meaningful but partial signal for estimating cost. While it cannot fully substitute for execution level metrics such as duration and memory, it offers a practical early indicator that could be used before a function has accumulated sufficient runtime data.

The identification and correction of the data leakage issue in the initial model is a central part of this project. Rather than reporting an artificially high accuracy score, the analysis was revised to reflect a methodologically sound evaluation, resulting in a lower but defensible and interpretable outcome.

6. Conclusion

This project analyzed a large scale, real world serverless computing dataset to understand invocation patterns and to build a predictive model for function level cost. Through careful data processing, feature engineering grounded in serverless billing principles, and rigorous model validation, the analysis produced a realistic estimate of how much cost can be explained by invocation behavior alone. The project reflects practical skills in handling large datasets under memory constraints, engineering domain relevant features, and applying machine learning methods with attention to validity and interpretability, skills directly applicable to research in cloud computing and distributed systems.

7. Dataset Reference

Shahrad, M., Fonseca, R., Goiri, I., Chaudhry, G., Batum, P., Cooke, J., Laureano, E., Tresness, C., Russinovich, M., and Bianchini, R. Serverless in the Wild: Characterizing and Optimizing the Serverless Workload at a Large Cloud Provider. Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC 20), USENIX Association, Boston, MA, July 2020.

Dataset repository: [Azure/AzurePublicDataset](https://github.com/Azure/AzurePublicDataset), available at github.com/Azure/AzurePublicDataset